

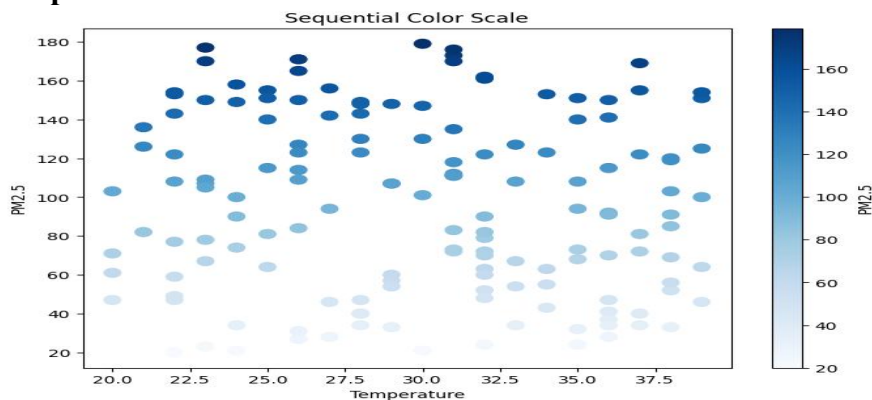
Activity-2
15-07-26

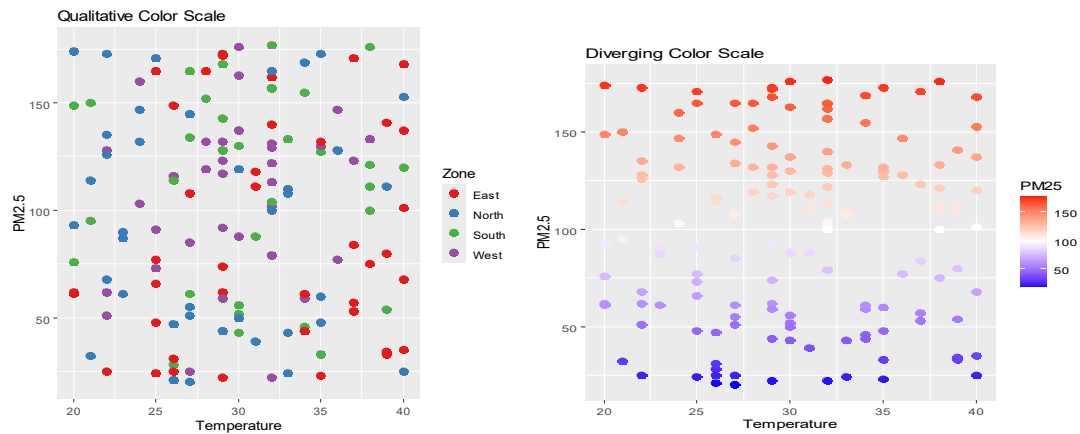
1. Smart City Air Quality Monitoring

Code:

R Program	Python Program
<pre>library(ggplot2) df <- data.frame(Temp = c(25, 26, 27, 28, 29), PM25 = c(40, 60, 80, 100, 120), Zone = c("A", "B", "C", "D", "E")) ggplot(df, aes(x = Temp, y = PM25, color = PM25)) + geom_point(size = 4) + scale_color_gradient(low = "lightblue", high = "blue") + labs(title = "Sequential Color Scale") ggplot(df, aes(x = Temp, y = PM25, color = PM25)) + geom_point(size = 4) + scale_color_gradient2(low = "blue", mid = "white", high = "red", midpoint = 80) + labs(title = "Diverging Color Scale") # Qualitative Color Scale ggplot(df, aes(x = Temp, y = PM25, color = Zone)) + geom_point(size = 4) + labs(title = "Qualitative Color Scale")</pre>	<pre>import matplotlib.pyplot as plt temp = [25,26,27,28,29] pm25 = [40,60,80,100,120] # Sequential plt.scatter(temp, pm25, c=pm25, cmap="Blues", s=100) plt.colorbar(label="PM2.5") plt.title("Sequential Color Scale") plt.show() # Diverging plt.scatter(temp, pm25, c=pm25, cmap="coolwarm", s=100) plt.colorbar(label="PM2.5") plt.title("Diverging Color Scale") plt.show() # Qualitative zone=[0,1,2,3,4] plt.scatter(temp, pm25, c=zone, cmap="Set1", s=100) plt.colorbar() plt.title("Qualitative Color Scale") plt.show()</pre>

Output:





Justification

- Sequential palette: Best for showing gradual changes in pollution levels.
- Diverging palette: Best for highlighting pollution above or below a critical threshold.
- Qualitative palette: Best for distinguishing different sensor locations or city zones, not pollution intensity.
- Most effective visualization: Diverging color scale, because it clearly separates safe and hazardous pollution levels, making pollution hotspots easy to identify.

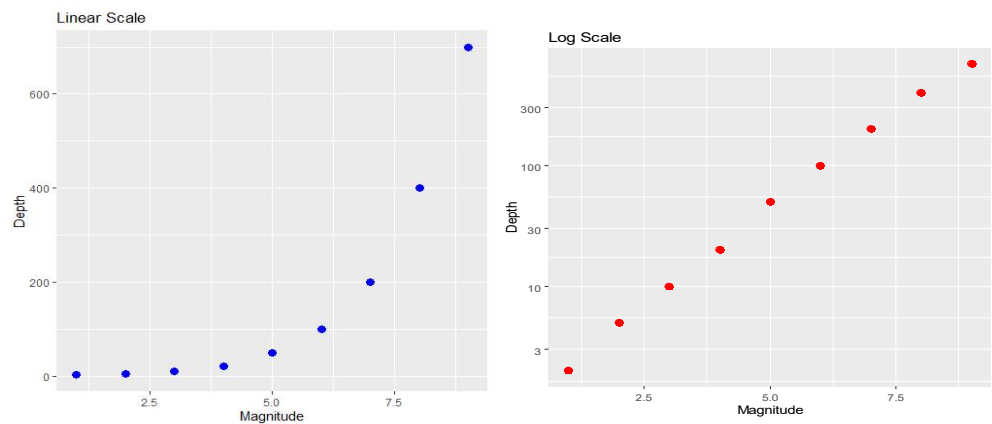
2. Earthquake Risk Analysis (Linear and Log Scale)

Code:

R Program	Python Program
<pre>library(ggplot2) df <- data.frame(Magnitude = c(1,2,3,4,5,6,7,8,9), Depth = c(2,5,10,20,50,100,200,400,700)) # Linear Scale ggplot(df, aes(Magnitude, Depth)) + geom_point(size=3, color="blue") + labs(title="Linear Scale") # Log Scale ggplot(df, aes(Magnitude, Depth)) +</pre>	<pre>import matplotlib.pyplot as plt mag=[1,2,3,4,5] depth=[5,20,80,200,700] # Linear plt.plot(mag, depth, marker="o") plt.title("Linear Scale") plt.xlabel("Magnitude") plt.ylabel("Depth") plt.show() # Log Scale plt.plot(mag, depth, marker="o")</pre>

<pre>geom_point(size=3, color="red") + scale_y_log10() + labs(title="Log Scale")</pre>	<pre>plt.yscale("log") plt.title("Log Scale") plt.xlabel("Magnitude") plt.ylabel("Depth") plt.show()</pre>
--	--

Output:



Justification:

- Linear scale: Large depth values dominate.
- Log scale: Small and large depth values become easier to compare.

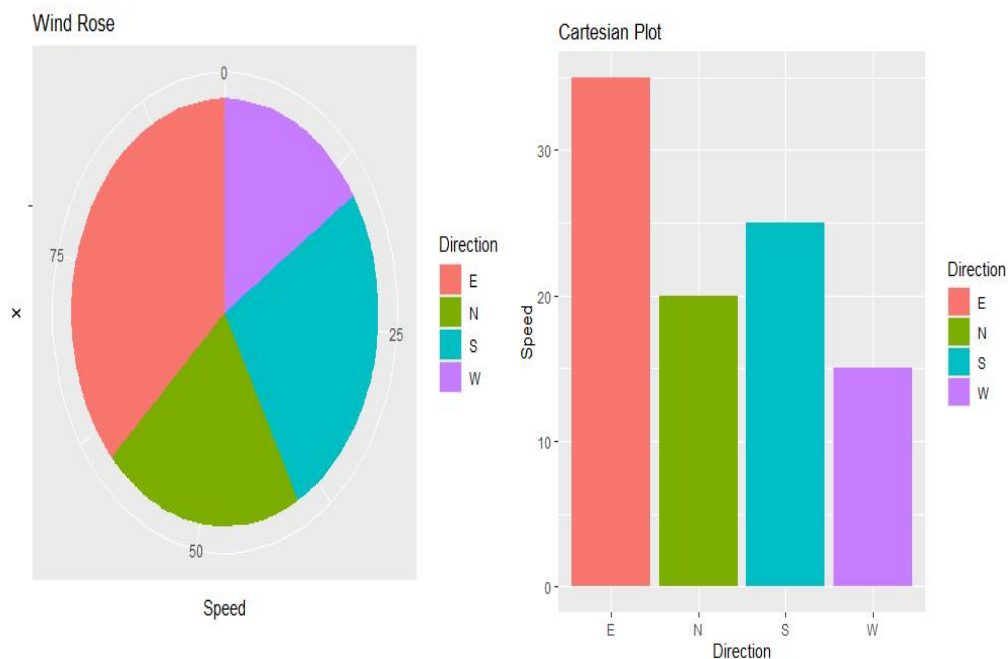
3.Wind Direction Analysis (Polar Coordinates & Cartesian Plot)

Code:

R Program	Python Program
<pre>library(ggplot2) df <- data.frame(Magnitude = c(1,2,3,4,5,6,7,8,9), Depth = c(2,5,10,20,50,100,200,400,700)) # Linear Scale ggplot(df, aes(Magnitude, Depth)) +</pre>	<pre>import numpy as np import matplotlib.pyplot as plt direction=["N","E","S","W"] speed=[20,35,25,15] # Cartesian Plot plt.bar(direction, speed, color="skyblue") plt.title("Cartesian Plot")</pre>

<pre>geom_point(size=3, color="blue") + labs(title="Linear Scale") # Log Scale ggplot(df, aes(Magnitude, Depth)) + geom_point(size=3, color="red") + scale_y_log10() + labs(title="Log Scale")</pre>	<pre>plt.xlabel("Direction") plt.ylabel("Speed") plt.show() # Polar Plot theta=np.linspace(0,2*np.pi,4,endpoint=False) plt.subplot(projection="polar") plt.bar(theta, speed) plt.title("Wind Rose") plt.show()</pre>
--	--

Output:



Justification:

- Polar plot (Wind Rose): Best for showing dominant wind directions.
- Cartesian plot: Best for comparing wind speeds only.
- Best visualization: Polar (Wind Rose) because it clearly represents wind direction, making it more useful for airport operations.

4. Hospital Patient Monitoring Dashboard (Color Highlighting)

Code:

R Program	Python Program
<pre>library(ggplot2) df <- data.frame(Patient = 1:5, HeartRate = c(72, 95, 120, 65, 140)) ggplot(df, aes(Patient, HeartRate, fill = HeartRate > 100)) + geom_col() + scale_fill_manual(values = c("green", "red")) + labs(title = "Heart Rate Monitoring")</pre>	<pre>import matplotlib.pyplot as plt patient = [1,2,3,4,5] heart = [72,95,120,80,140] colors = ["green" if x <= 100 else "red" for x in heart] plt.bar(patient, heart, color=colors) plt.title("Hospital Patient Monitoring") plt.xlabel("Patient") plt.ylabel("Heart Rate") plt.show()</pre>

Output:



Justification:

- Green: Normal heart rate (≤ 100 bpm).
- Red: High heart rate (> 100 bpm), immediately attracts attention.

5. Global Climate Change Study (Heatmaps with Diverging Color Palettes)

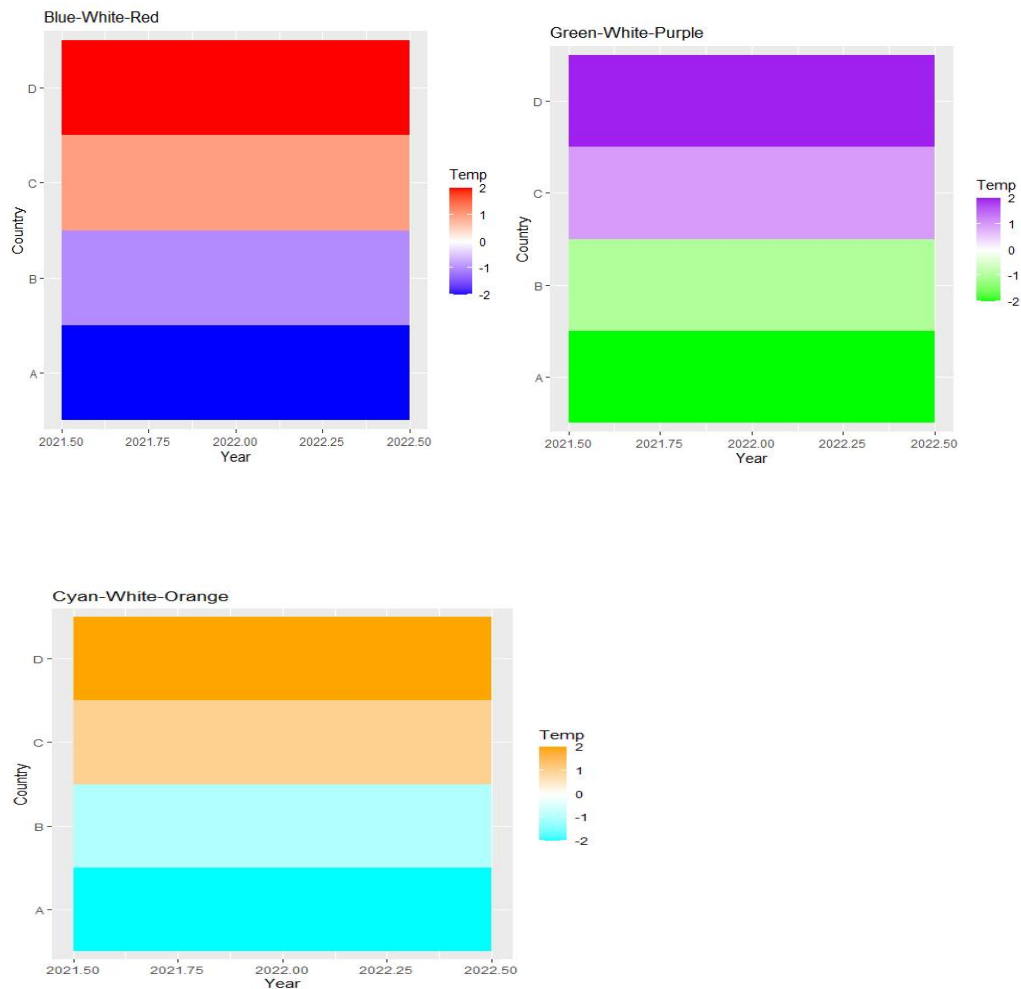
Code:

R Program	Python Program
<pre>library(ggplot2) df <- data.frame(Country = c("A","B","C","D"), Year = c(2022,2022,2022,2022), Temp = c(-2, -1, 1, 2)) # Palette 1: Blue-White-Red ggplot(df, aes(Year, Country, fill = Temp)) + geom_tile() + scale_fill_gradient2(low = "blue", mid = "white", high = "red") + labs(title = "Blue-White-Red") # Palette 2: Green-White-Purple ggplot(df, aes(Year, Country, fill = Temp)) + geom_tile() + scale_fill_gradient2(low = "green", mid = "white", high = "purple") + labs(title = "Green-White-Purple") # Palette 3: Cyan-White-Orange ggplot(df, aes(Year, Country, fill = Temp)) + geom_tile() +</pre>	<pre>import matplotlib.pyplot as plt data = [[-2,-1], [1,2]] # Palette 1 plt.imshow(data, cmap="coolwarm") plt.colorbar() plt.title("Blue-White-Red") plt.show() # Palette 2 plt.imshow(data, cmap="PiYG") plt.colorbar() plt.title("Green-Purple") plt.show() # Palette 3 plt.imshow(data, cmap="bwr") plt.colorbar() plt.title("Red-Blue") plt.show()</pre>

```
scale_fill_gradient2(low = "cyan", mid =
"white", high = "orange") +

labs(title = "Cyan-White-Orange")
```

Output:



Comparison

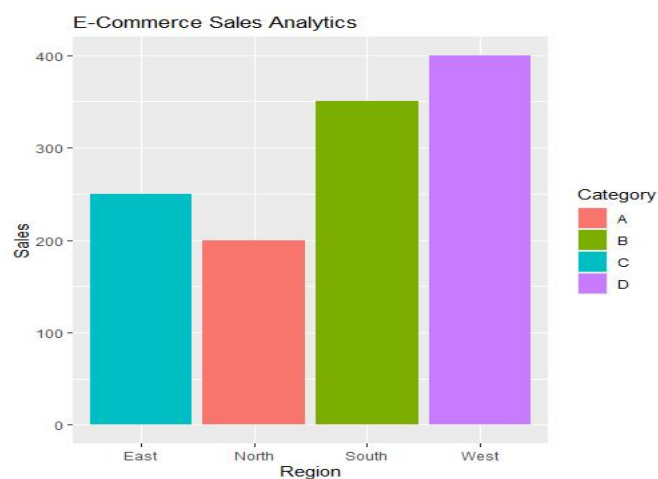
- Blue-White-Red: Best for showing negative (blue) and positive (red) temperature anomalies.
- Green-White-Purple: Distinguishes values but is less intuitive.
- Cyan-White-Orange: Good contrast but less commonly used.
- Best palette: Blue-White-Red, because it clearly represents cooler (negative) and warmer (positive) anomalies and is easy to interpret.

6. E-Commerce Sales Analytics

Code:

R Program	Python Program
<pre>library(ggplot2) df <- data.frame(Region = c("North","South","East","West"), Sales = c(200,350,250,400), Category = c("A","B","C","D")) # Categories + Sales ggplot(df, aes(x = Region, y = Sales, fill = Category)) + geom_col() + labs(title = "E-Commerce Sales Analytics")</pre>	<pre>import matplotlib.pyplot as plt region = ["North","South","East","West"] sales = [200,350,250,400] colors = ["red","blue","green","orange"] plt.bar(region, sales, color=colors) plt.title("E-Commerce Sales") plt.xlabel("Region") plt.ylabel("Sales") plt.show()</pre>

Output:



Discussion:

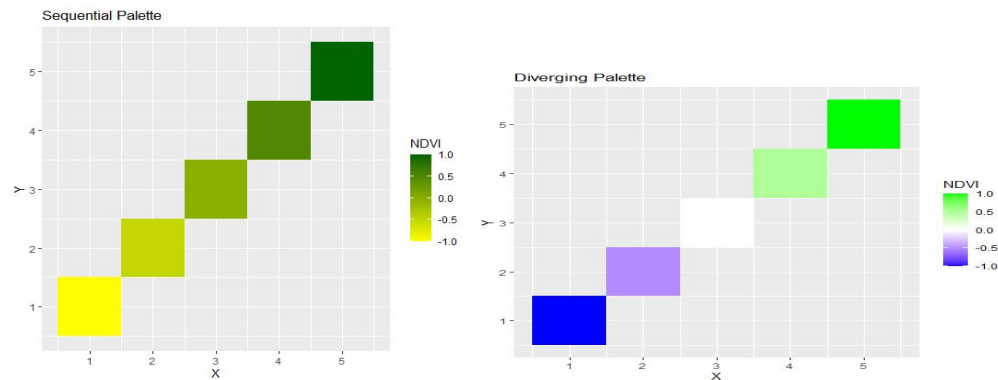
- Different colors distinguish product categories.
- Bar height represents sales volume.
- Using color for both category and value at the same time can confuse users, so it is better to use color for categories and bar height for sales.

7. Satellite Image Vegetation Analysis

Code:

R Program	Python Program
<pre>library(ggplot2) df <- data.frame(X = 1:5, Y = 1:5, NDVI = c(-1, -0.5, 0, 0.5, 1)) # Sequential Palette ggplot(df, aes(X, Y, fill = NDVI)) + geom_tile() + scale_fill_gradient(low = "yellow", high = "darkgreen") + labs(title = "Sequential Palette") # Diverging Palette ggplot(df, aes(X, Y, fill = NDVI)) + geom_tile() + scale_fill_gradient2(low = "blue", mid = "white", high = "green", midpoint = 0) + labs(title = "Diverging Palette")</pre>	<pre>import matplotlib.pyplot as plt ndvi = [[-1,-0.5], [0.5,1]] # Sequential Palette plt.imshow(ndvi, cmap="Greens") plt.colorbar() plt.title("Sequential Palette") plt.show() # Diverging Palette plt.imshow(ndvi, cmap="RdYlGn") plt.colorbar() plt.title("Diverging Palette") plt.show()</pre>

Output:



Evaluation

- Sequential palette: Best for showing increasing vegetation health.
- Diverging palette: Clearly separates water (blue), barren land (white), and healthy vegetation (green).
- Best choice: Diverging palette, because it reduces interpretation errors and makes negative, neutral, and positive NDVI values easy to distinguish.

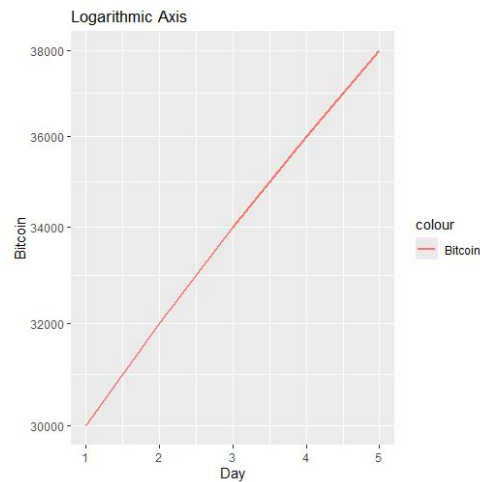
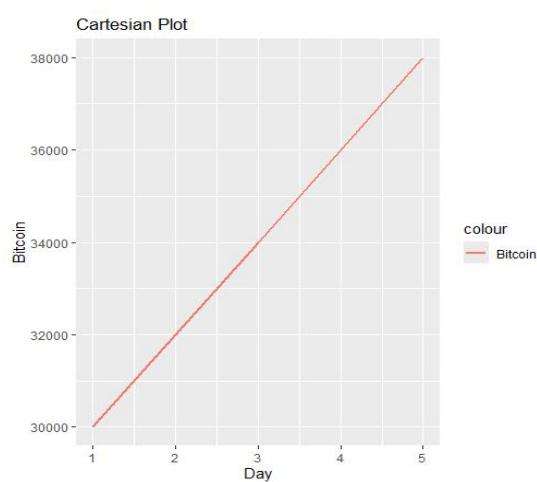
8. Cryptocurrency Market Volatility

Code:

R Program	Python Program
<pre>library(ggplot2) df <- data.frame(Day = 1:5, Bitcoin = c(30000,32000,34000,36000,38000)) ggplot(df, aes(Day, Bitcoin, color = "Bitcoin")) + geom_line(linewidth = 1) + labs(title = "Cartesian Plot") ggplot(df, aes(Day, Bitcoin, color = "Bitcoin")) + geom_line(linewidth = 1) +</pre>	<pre>import matplotlib.pyplot as plt day = [1,2,3,4,5] bitcoin = [30000,32000,34000,36000,38000] # Cartesian Plot plt.plot(day, bitcoin, color="blue") plt.title("Cartesian Plot") plt.xlabel("Day") plt.ylabel("Price") plt.show() # Logarithmic Axis</pre>

<pre>scale_y_log10() + labs(title = "Logarithmic Axis")</pre>	<pre>plt.plot(day, bitcoin, color="red") plt.yscale("log") plt.title("Logarithmic Axis") plt.xlabel("Day") plt.ylabel("Price") plt.show()</pre>
---	---

Output:



Result:

- Cartesian: Shows actual prices.
- Log scale: Makes cryptocurrencies with very different price ranges easier to compare.

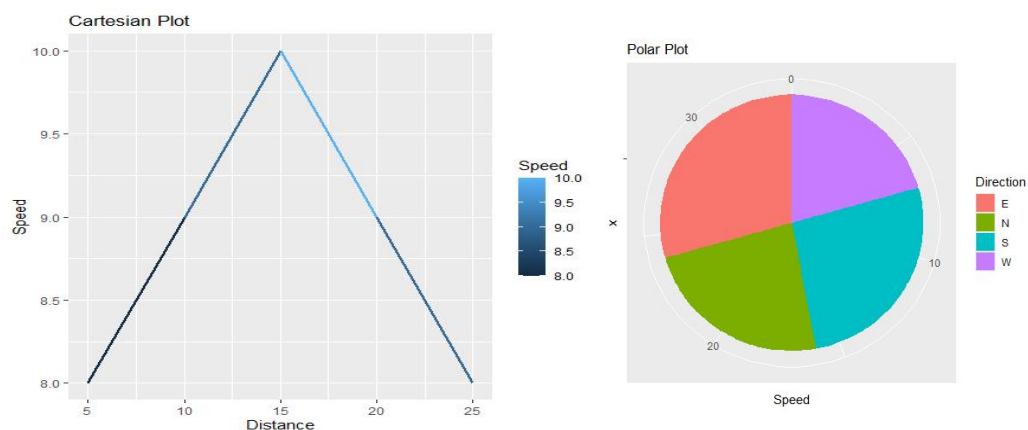
9. Marathon Runner Performance Analysis

Code:

R Program	Python Program
<pre>library(ggplot2) df <- data.frame(Distance = c(5,10,15,20,25), Speed = c(8,9,10,9,8))</pre>	<pre>import matplotlib.pyplot as plt import numpy as np distance = [5,10,15,20,25] speed = [8,9,10,9,8]</pre>

<pre> ggplot(df, aes(Distance, Speed, color = Speed)) + geom_line(linewidth = 1) + labs(title = "Cartesian Plot") df2 <- data.frame(Direction = c("N","E","S","W"), Speed = c(8,10,9,7)) ggplot(df2, aes(x = "", y = Speed, fill = Direction)) + geom_bar(stat = "identity", width = 1) + coord_polar(theta = "y") + labs(title = "Polar Plot") </pre>	<pre> # Cartesian Plot plt.scatter(distance, speed, c=speed, cmap="viridis", s=100) plt.colorbar(label="Speed") plt.title("Cartesian Plot") plt.xlabel("Distance") plt.ylabel("Speed") plt.show() # Polar Plot theta = np.linspace(0, 2*np.pi, 5, endpoint=False) plt.subplot(projection="polar") plt.scatter(theta, speed, c=speed, cmap="viridis", s=100) plt.title("Polar Plot") plt.show() </pre>
---	---

Output:



Result:

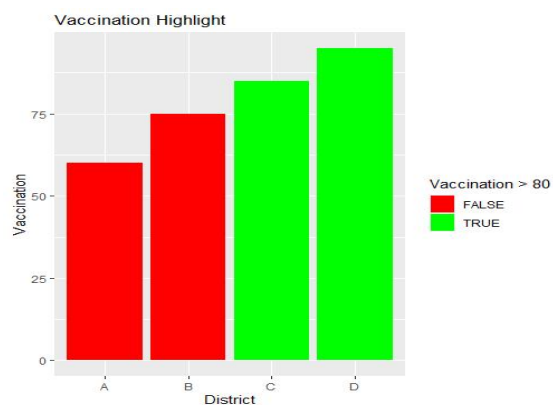
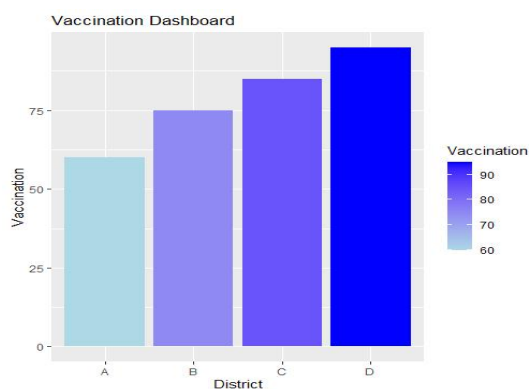
- Cartesian: Best for speed over distance.
- Polar: Best for showing running direction.

10. COVID-19 Vaccination Campaign Dashboard

Code:

R Program	Python Program
<pre>library(ggplot2) df <- data.frame(District = c("A","B","C","D"), Vaccination = c(60,75,85,95)) ggplot(df, aes(District, Vaccination, fill = Vaccination)) + geom_col() + scale_fill_gradient(low = "lightblue", high = "blue") + labs(title = "Vaccination Dashboard") ggplot(df, aes(District, Vaccination, fill = Vaccination > 80)) + geom_col() + scale_fill_manual(values = c("red", "green")) + labs(title = "Vaccination Highlight")</pre>	<pre>import matplotlib.pyplot as plt district = ["A","B","C","D"] vaccination = [60,75,85,95] # Version 1 - Blue Gradient plt.bar(district, vaccination, color=["lightblue","skyblue","blue","navy"]) plt.title("Vaccination Dashboard") plt.show() # Version 2 - Highlighting colors = ["red" if x < 80 else "green" for x in vaccination] plt.bar(district, vaccination, color=colors) plt.title("Vaccination Highlight") plt.show()</pre>

Output:



Evaluation

- Blue gradient: Suitable for policymakers to compare vaccination rates.
- Green/Red highlighting: Easy for the public to identify high and low coverage (though avoid relying only on red/green for users with red–green color vision deficiency; adding labels or patterns improves accessibility).
- Most effective: Blue gradient with labels is generally more accessible and easier to interpret across a wider audience.